

Generative AI, kept behind a verifier

Step 9, the bonus. SHAP values a tutor cannot read, turned into a note a tutor can act on.

Jeryl Donato Estopace

PGD AI/ML capstone, Pillar 5

The explanation layer for the dropout model, built against a baseline and behind a check

SHAP values a tutor cannot read

The one thing a language model is genuinely good for here, done the way the rest of the project works.

Step 5 produces a SHAP value like **Tuition fees up to date: +0.68**. True, and useless to the tutor who picks up the phone. So a language model turns a student's top drivers into two plain sentences.

Two ways to do it, and both get built. A rules engine, a lookup table from each feature to a clause and an action, deterministic and unable to invent anything. And the language model. The honest question is what the model bought that the table did not.

The table stays silent where it should. It has no row for a course code or a parent's occupation, because neither is a fact a tutor can act on. That silence is the gap the model has to earn its place by filling.

56%

of driver slots the rules table can voice

The rest is Course and a parent's occupation, the model's strongest and least sayable signals. Fill them with something true, or invent.

A prompt is a request. The verifier is the control.

The guardrail is the deliverable, not the API call.

The guarded prompt bans four things: numbers, scoring words, any driver not in the student's own list, and gender, though the model leans on it at SHAP 0.301. Then a cell checks the prompt actually held.

To prove the check does something, the task runs twice. A naive prompt, and the guarded one. The naive prompt printed probabilities and admission scores, read a mother's occupation as low status, and named the student's gender. The guarded prompt did none of it.

The rules-engine baseline passes by construction. It has no mechanism for saying a thing it was not given. The language model has to be watched.

0 / 5
naive prompt safe

5 / 5
guarded prompt safe

If the guarded prompt regressed tomorrow, the verifier fails loudly instead of quietly shipping a sentence with a number in it. That is the difference between using a language model and deploying one.

What it bought, and what it did not

The context is the work. The generation is the easy part.

It bought fluency, and words for the course code and the parent's occupation the table refuses to touch.

It also bought one note that over-reached. A guarded note turned a parent's occupation into 'less familiarity with higher education', an inference the table declined to make and one the verifier is not built to catch. The verifier is necessary, not sufficient.

Everything the model knows about which number matters, it knows because it was told, in a context block that took the rest of the project to earn. And the whole notebook runs, and renders, for a grader with no key, off a committed cache of every prompt and every response.

Generative AI did not replace Step 5. It sits on top of it, and it is only trustworthy because Step 5 came first.